

EXERCISE 3 · MID DECK 2 · ANSWER KEY

Tool Use and Knowledge Bases

Apply the tool-use loop and RAG pipeline — including the classic gotchas.

6 questions · For trainer use · Distribute after the exercise

HOW TO USE

Walk through each answer with the cohort. The explanation is what matters — many of these have answers that look right but fail for subtle reasons.

Q1 OF 6 · CONCEPT

In the 8-step tool-use flow, why does the deck emphasize that the model **never executes** the tool?

- A. Because the model would slow down if it had to run code.
- B. Because the model returns a structured request asking *your code* to run the tool, then your code sends the result back.
- C. Because AWS prohibits the model from making outbound network calls.
- D. Because tool execution requires a separate IAM permission the model doesn't have.

ANSWER

B.

This is the architectural separation that makes tool use safe and predictable. The model only emits a request (the toolUse block). Your code controls what actually runs, validates inputs, applies auth, and decides whether to even honor the request. The model never crosses into your zone.

Q2 OF 6 · BUG SPOT

This tool-use code returns a valid response on the first API call but the second API call returns *ValidationException*. What did the developer forget?

```
if stop_reason == "tool_use":
    tool_use_block = response["output"]["message"]["content"][0]["toolUse"]
    result = run_tool(tool_use_block["name"], tool_use_block["input"])

    messages.append({
        "role": "user",
        "content": [{
            "toolResult": {
                "toolUseId": tool_use_block["toolUseId"],
                "content": [{"json": result}],
            }
        }]
    })

    final = client.converse(
        modelId=model_id,
        messages=messages,
        toolConfig=TOOL_CONFIG,
    )
```

ANSWER

The assistant's tool-use message is never appended to `messages` before the second call.

Bedrock expects this sequence in messages: user → assistant (with toolUse) → user (with toolResult). The code skips the assistant message, so the toolResult appears to come out of nowhere. Add `messages.append(response["output"]["message"])` before appending the toolResult.

Q3 OF 6 · CONCEPT

Why does semantic search find a match between "refund window" and "customers can return items within 30 days", when keyword search does not?

- A. Semantic search uses regular expressions to find synonyms.
- B. Both phrases are converted to vectors by an embedding model; vectors with similar meaning are close in vector space.
- C. Bedrock maintains a built-in synonym dictionary that maps "refund" to "return".
- D. Semantic search uses fuzzy string matching with a Levenshtein distance threshold.

ANSWER

B.

Embeddings convert text to numerical vectors. The embedding model is trained so that text with similar meaning produces vectors that are close together by cosine or Euclidean distance. There's no synonym table, no regex, no fuzzy matching — just vector arithmetic.

Q4 OF 6 · BUG SPOT

A learner is querying a Knowledge Base. The code below throws `AttributeError: 'BedrockRuntime' object has no attribute 'retrieve_and_generate'`. Why?

```
import boto3

client = boto3.client("bedrock-runtime", region_name="us-east-1")

response = client.retrieve_and_generate(
    input={"text": "When is spring break?"},
    retrieveAndGenerateConfiguration={
        "type": "KNOWLEDGE_BASE",
        "knowledgeBaseConfiguration": {
            "knowledgeBaseId": "ABC123",
            "modelArn": "us.amazon.nova-lite-v1:0",
        },
    },
)
```

ANSWER

Wrong client. Knowledge Bases use `bedrock-agent-runtime`, not `bedrock-runtime`.

`retrieve_and_generate` is part of the agent runtime API surface, not the model runtime. Fix the client to `boto3.client("bedrock-agent-runtime", region_name="us-east-1")`. This is one of the most common mistakes when moving from plain Converse to RAG.

Q5 OF 6 · CONCEPT

In the 5-step RAG pipeline, which steps happen **once during setup** and which happen **on every user query**?

- A. All five run on every query.
- B. Steps 1-3 (upload, embed, store) run once; steps 4-5 (retrieve, generate) run on every query.
- C. Steps 1 and 2 run once; steps 3-5 run on every query.
- D. All five run once during setup; nothing runs at query time.

ANSWER

B.

This split matters operationally. Ingestion (upload → chunk → embed → store) is a one-time (or batch-refresh) cost. Retrieval and generation run on every user question. When pricing a RAG system, the recurring cost is dominated by steps 4-5.

Q6 OF 6 · DESIGN SCENARIO

Your team built a tool-use loop where the model can call `get_weather`, `get_stock_price`, and `send_email`.

Name **one risk** in this setup and **one mitigation** you'd add before shipping.

ANSWER

Multiple valid answers.

*Strong answers: (1) **Risk**: model might call `send_email` when not asked to, or based on a manipulated prompt.*

***Mitigation**: require user confirmation for any state-changing tool; treat email as a side effect, not a free action. (2) **Risk**: bad tool input crashes your code. **Mitigation**: validate `tool_input` against the schema in `run_tool` before executing.*

*(3) **Risk**: model invents tool names that don't exist. **Mitigation**: `run_tool` returns a structured error the model can recover from. (4) **Risk**: missing `toolUseId` in the result. **Mitigation**: assert it's present.*